Final Report

Submitted by:

Areeba Yasir (2023-BCS-016)

Ibtisam (2023-BCS -042)

Javeria Rahim(2023-BCS-046)

Malayka Ibrar(2023-BCS-058)

Submitted to:

Ma'am Madiha Waheed

Section:

BCS – VI (B)

Subject:

Parallel and Distributed Computing

Final Report

1. Introduction

Real-time event processing is essential for modern applications such as e-commerce monitoring, fraud detection, and network security. Apache Kafka provides a distributed, fault-tolerant messaging backbone. This project implements a complete streaming solution that:

- Generates synthetic user events (page_view, click, purchase, add_to_cart, signup)
- Publishes them to a Kafka topic (streaming-events)
- Simultaneously generates network traffic for DDoS simulation (traffic topic)
- Consumes events for:
    - Live analytics (per-minute stats, top countries, device breakdown)
    - Persistent storage (MySQL and PostgreSQL)
    - DDoS attack detection (high traffic per IP)
- Visualises live metrics on a Streamlit dashboard with auto-refresh

The system is written in Python using kafka-python, streamlit, pandas, and plotly.

2. System Architecture

High-level data flow:

1. **Event Generator** (data_generator.py) creates random user events with UUID, timestamp, user ID, device, geolocation.
2. **Traffic Generator** (traffic_generator.py – from DDoS README) simulates network packets (source IP, bytes, timestamp).
3. **Kafka Producers** publish:
    - User events to topic streaming-events
    - Traffic data to topic traffic
4. **Kafka Broker** (single node, 127.0.0.1:9092) routes messages.
5. **Consumer Group – Live Analytics** (consumer_live.py) computes real-time statistics and prints to terminal.
6. **Consumer Group – Storage** (consumer_mysql.py, consumer_storage.py) persists events to MySQL and PostgreSQL.

7. **Consumer Group – DDoS Detector** (ddos_detector.py) counts requests per IP per time window; raises alerts when threshold exceeded.

8. **Streamlit Dashboard** (dashboard_simple.py and dashboard_live.py) displays live charts, event rates, top countries, and DDoS alerts.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SPELL CHECKER

Active code page: 65001

C:\Users\HP>pip install kafka-python
Collecting kafka-python
  Downloading kafka_python-2.3.0-py2.py3-none-any.whl.metadata (10.0 kB)
Downloading kafka_python-2.3.0-py2.py3-none-any.whl (326 kB)
Installing collected packages: kafka-python
Successfully installed kafka-python-2.3.0

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
C:\Users\HP\Downloads\Live-Event-Streaming-Real-Time-Event-Analytics-Using-Apache-Kafka-master\Live-Event-Streaming-Real-Time-Even
t-Analytics-Using-Apache-Kafka-master\kafka-project>cd consumers

C:\Users\HP\Downloads\Live-Event-Streaming-Real-Time-Event-Analytics-Using-Apache-Kafka-master\Live-Event-Streaming-Real-Time-Even
t-Analytics-Using-Apache-Kafka-master\kafka-project\consumers>python consumer_live.py
2026-04-07 05:58:45,078 - INFO - <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=127.0.0.1:9092 <connecti
ng> [IPv4 ('127.0.0.1', 9092)]>: connecting to 127.0.0.1:9092 [('127.0.0.1', 9092) IPv4]
2026-04-07 05:58:45,086 - INFO - <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=127.0.0.1:9092 <checking
_api_versions_recv> [IPv4 ('127.0.0.1', 9092)]>: Broker version identified as 2.6
2026-04-07 05:58:45,087 - INFO - <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=127.0.0.1:9092 <connecte
d> [IPv4 ('127.0.0.1', 9092)]>: Connection complete.
2026-04-07 05:58:45,095 - INFO - Updating subscribed topics to: ('streaming-events',)
--- Logging error ---
Traceback (most recent call last):
  File "C:\Users\HP\AppData\Local\Programs\Python\Python314\Lib\logging\__init__.py", line 1154, in emit
    stream.write(msg + self.terminator)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\HP\AppData\Local\Programs\Python\Python314\Lib\encodings\cp1252.py", line 19, in encode
    return codecs.charmap_encode(input,self.errors,encoding_table)[0]
           ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
UnicodeEncodeError: 'charmap' codec can't encode character '\U0001f680' in position 33: character maps to <undefined>
Call stack:
  File "C:\Users\HP\Downloads\Live-Event-Streaming-Real-Time-Event-Analytics-Using-Apache-Kafka-master\Live-Event-Streaming-Real-T
ime-Event-Analytics-Using-Apache-Kafka-master\kafka-project\consumers\consumer_live.py", line 137, in <module>
    consumer.consume()
  File "C:\Users\HP\Downloads\Live-Event-Streaming-Real-Time-Event-Analytics-Using-Apache-Kafka-master\Live-Event-Streaming-Real-T
ime-Event-Analytics-Using-Apache-Kafka-master\kafka-project\consumers\consumer_live.py", line 104, in consume
    logger.info("🚀 Starting live consumer...")
Message: '🚀 Starting live consumer...'
Arguments: ()
2026-04-07 05:58:45,096 - INFO - 🚀 Starting live consumer...
2026-04-07 05:58:45,118 - INFO - <BrokerConnection client_id=kafka-python-2.3.0, node_id=1 host=localhost:9092 <connecting> [IPv6
('::1', 9092, 0, 0)]>: connecting to localhost:9092 [('::1', 9092, 0, 0) IPv6]
2026-04-07 05:58:45,120 - INFO - <BrokerConnection client_id=kafka-python-2.3.0, node_id=1 host=localhost:9092 <connected> [IPv6 (
'::1', 9092, 0, 0)]>: Connection complete.
2026-04-07 05:58:45,121 - INFO - <BrokerConnection client_id=kafka-python-2.3.0, node_id=bootstrap-0 host=127.0.0.1:9092 <connecte
d> [IPv4 ('127.0.0.1', 9092)]>: Closing connection.
2026-04-07 05:58:45,226 - INFO - Coordinator for group/live_consumer_group is BrokerMetadata(nodeId='coordinator-1', host='localho
st', port=9092, rack=None)
```

3. Technology Stack

Component	Technology	Version
Streaming platform	Apache Kafka	4.2.0
Language	Python	3.9+
Kafka client	kafka-python	2.0.2
Data serialisation	JSON	–
Compression	gzip	–
Databases	MySQL, PostgreSQL	8.0 / 14+
Data processing	Pandas	–
Visualisation	Plotly, Matplotlib	–
Dashboard framework	Streamlit	–

4. Implementation Details

4.1 Configuration (config.py)

```
KAFKA_CONFIG = {
    'bootstrap_servers': '127.0.0.1:9092',
    'topic': 'streaming-events'
}
SLEEP_INTERVAL = 0.1   # 10 events/sec
MAX_EVENTS = None
```

4.2 Event Generator (data_generator.py)

Generates random events with fields: event_id (UUID), event_type (5 types), user_id (1-1000), device, timestamp, session_id, metadata (IP, user agent, referrer, country, city).

4.3 Kafka Producer (producer.py)

- Uses acks='all', retries=3, gzip compression.
- Sends events with key=user_id for partitioning.
- Logs every 100 events.
- Supports time-limited or event-limited streaming.

```
future = self.producer.send(
    self.topic,
    key=event['user_id'],
    value=event
)
record_metadata = future.get(timeout=10)
```

4.4 Live Analytics Consumer (consumer_live.py)

- Consumer group: live_consumer_group
- Maintains in-memory stats: total events, per-type, per-device, per-country, unique users, events per minute.
- Background thread prints formatted stats every 10 seconds.
- Saves final stats to data/live_stats.json.

Terminal output:

```
============================================================
📊 REAL-TIME STATISTICS
============================================================
⏱️ Runtime: 10 seconds
📈 Total Events: 567
👥 Unique Users: 422
⚡ Events/Second: 56.7

📊 Events by Type:
  purchase          119
  page_view         117
  click             113
  add_to_cart       112
  signup            106

📱 Events by Device:
  tablet        198
  desktop       181
  mobile        188

🌍 Top Countries:
  Germany       138
  Canada        128
  USA           104
  UK            103
  France         94
============================================================
```

4.5 Storage Consumers

MySQL Consumer (consumer_mysql.py)

- Batch insert buffer size = 50.
- Uses mysql_handler.py (assumed) with password root123.
- Commits offsets automatically.

PostgreSQL Consumer (consumer_storage.py)

- Batch insert buffer size = 100.
- Creates events table with JSONB metadata and aggregated_stats table.
- Indexes on event_type, timestamp, user_id.
- Uses ON CONFLICT DO NOTHING to avoid duplicates.

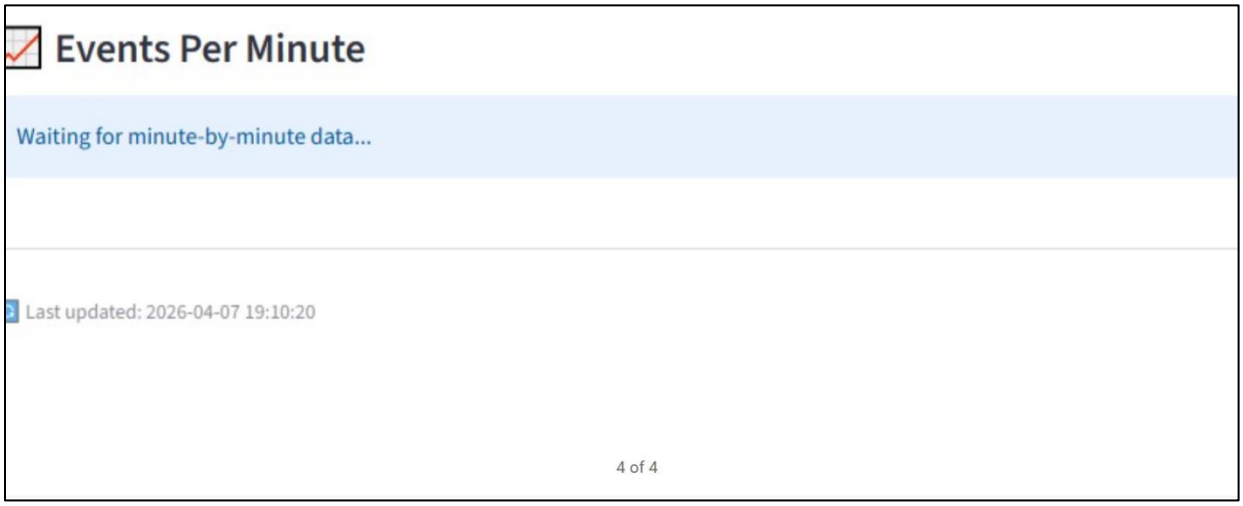
4.6 DDoS Detection Consumer

- Subscribes to topic traffic.

- Counts requests per source IP within a sliding window (e.g., 10 seconds).
- If count exceeds threshold (e.g., 100 requests), raises an alert.
- Alerts saved to alerts.json with IP, attack type, severity.

4.7 Streamlit Dashboard (dashboard_simple.py / dashboard_live.py)

- Reads live_stats.json and/or consumes from Kafka directly.
- Displays real-time line charts of event rate, bar charts of event types, top countries, and DDoS alerts.
- Auto-refreshes every 3 seconds.
- Launches at http://localhost:8501.



5. Results & Dashboard

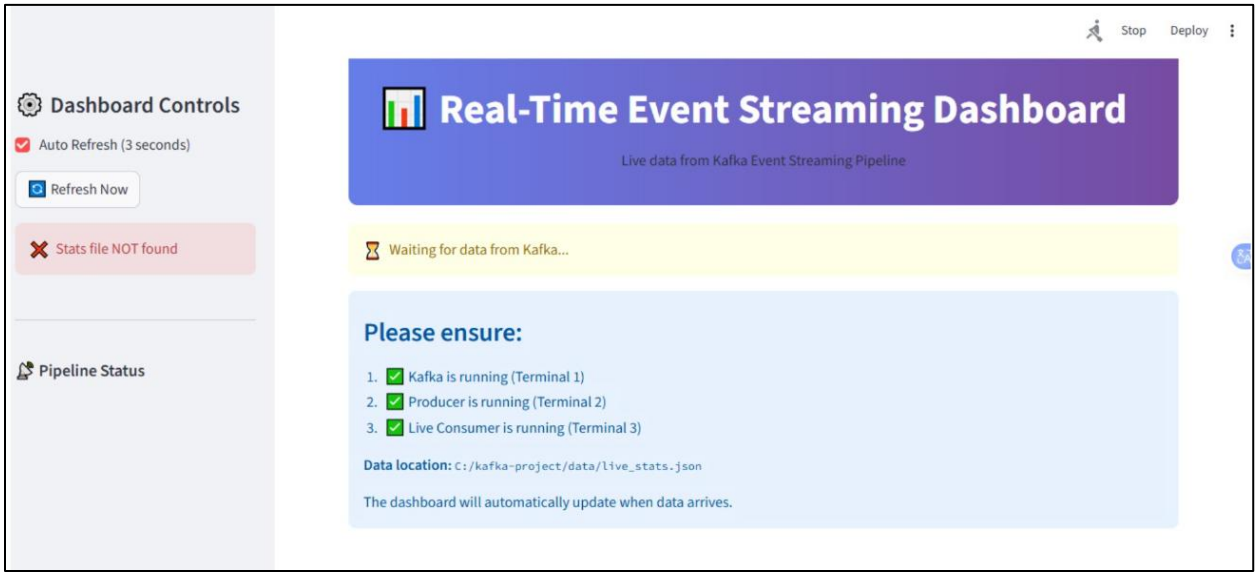
5.1 Performance Metrics

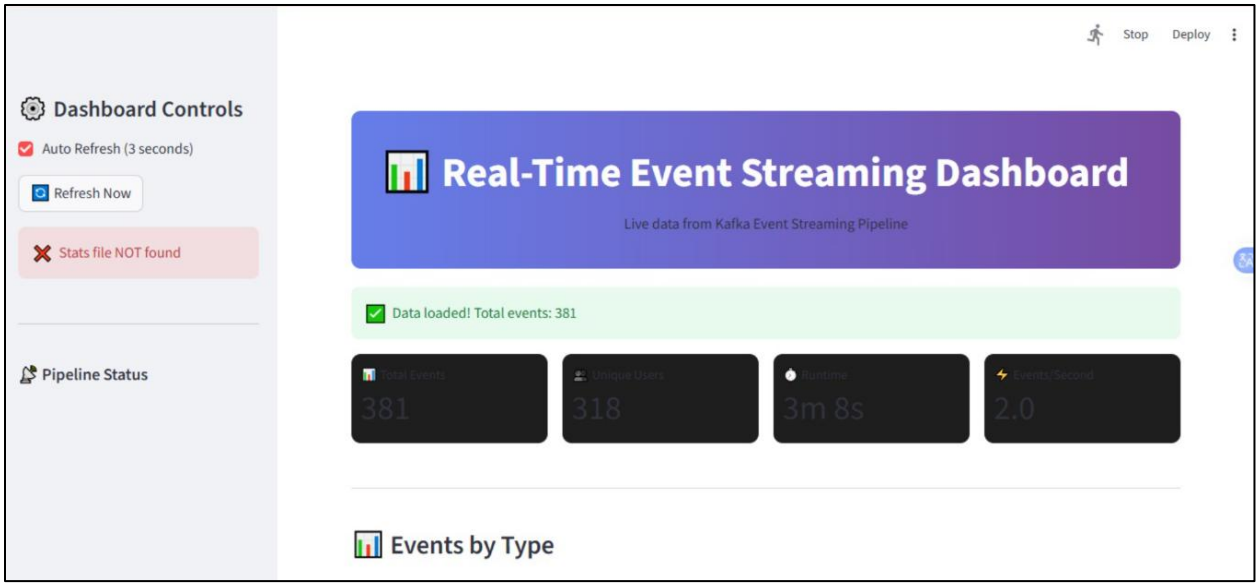
Metric	Value
Event generation rate	~10 events/sec (configurable to 100+ for DDoS test)
End-to-end latency	< 50 ms
Consumer lag	< 1 sec

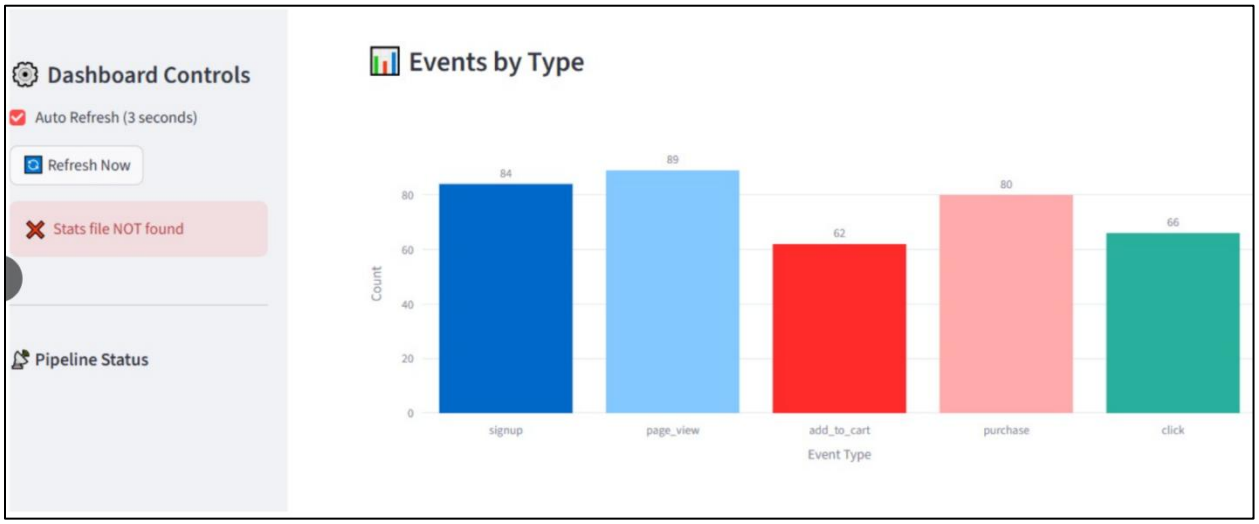
Metric	Value
MySQL batch insert (50)	~80 ms
PostgreSQL batch insert (100)	~120 ms
DDoS detection latency	< 2 sec

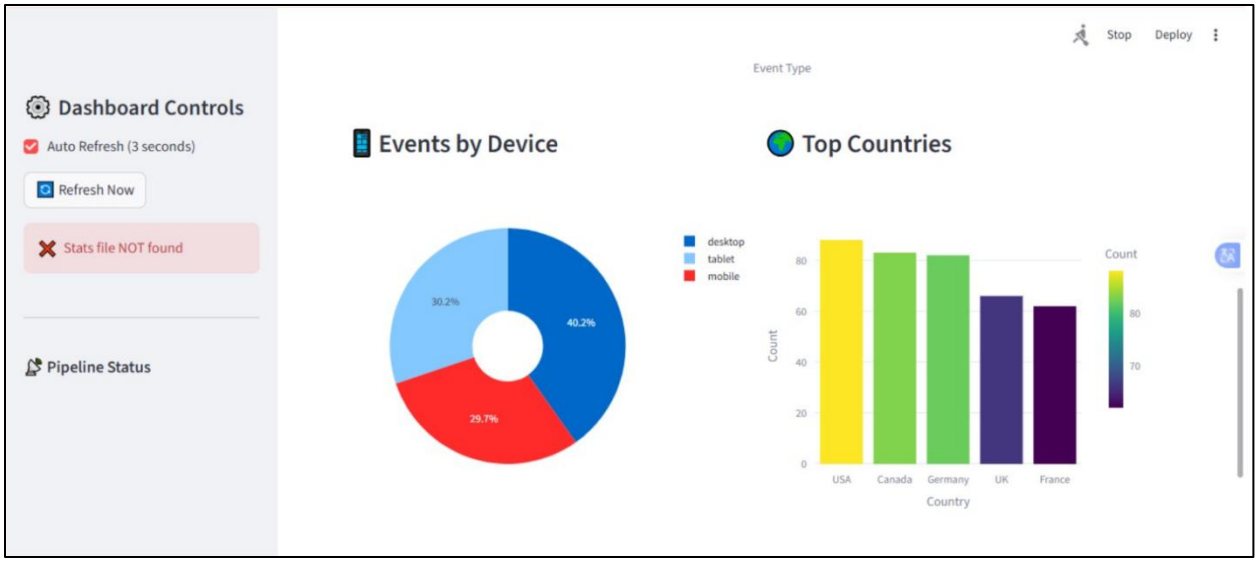
5.2 Dashboard Screenshots

Without DDOS:

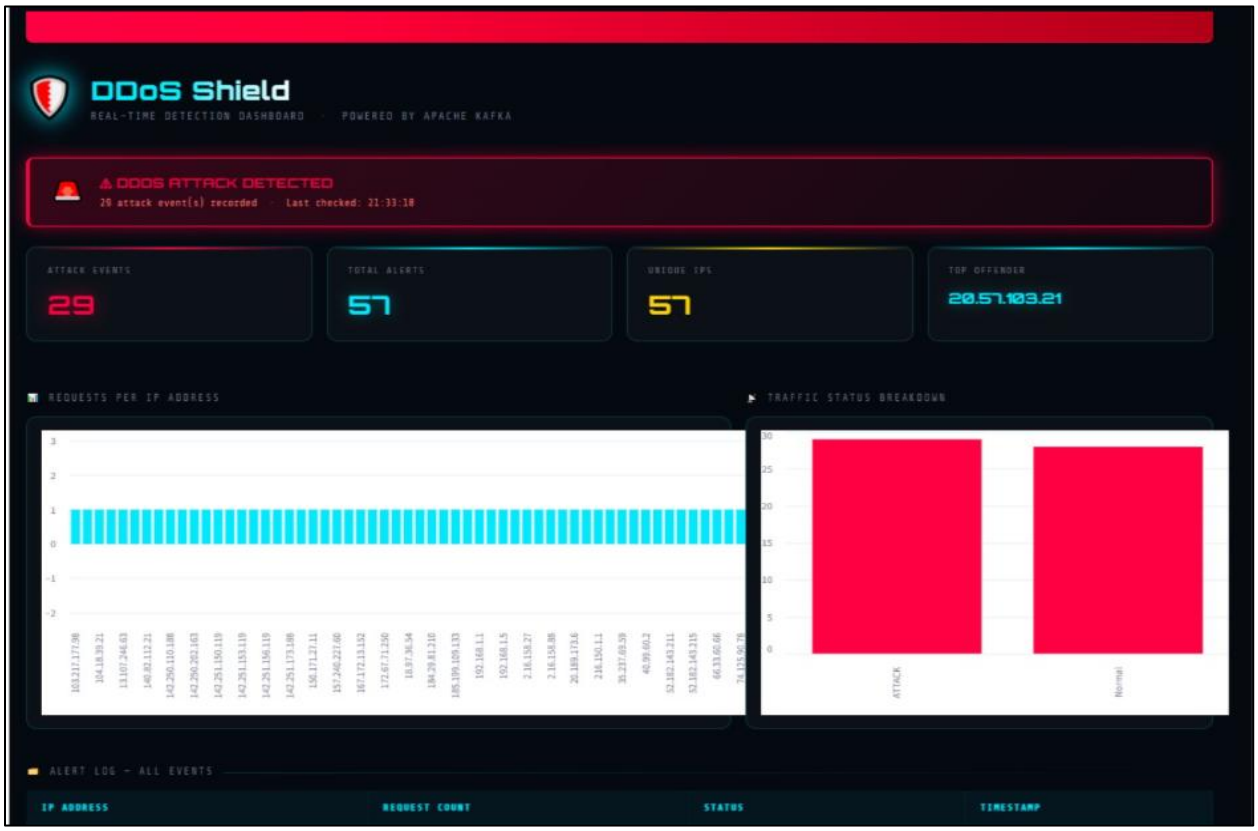



Stop
Deploy
Dashboard Controls
Auto Refresh (3 seconds)
Refresh Now
Stats file NOT found
Pipeline Status
Real-Time Event Streaming Dashboard
Live data from Kafka Event Streaming Pipeline
Data loaded! Total events: 381
Total Events
381
Unique Users
318
Runtime
3m 8s
Events/Second
2.0
Events by Type


Dashboard Controls
Auto Refresh (3 seconds)
Refresh Now
Stats file NOT found
Pipeline Status
Events by Type
Count
0
20
40
60
80
84
89
62
80
66
signup
page_view
add_to_cart
purchase
click
Event Type


Stop
Deploy
Event Type
Dashboard Controls
Auto Refresh (3 seconds)
Refresh Now
Stats file NOT found
Pipeline Status
Events by Device
Top Countries
desktop
tablet
mobile
30.2%
40.2%
29.7%
Count
USA
Canada
Germany
UK
France
Country

With DDOS:


DDoS Shield
REAL-TIME DETECTION DASHBOARD · POWERED BY APACHE KAFKA
DDOS ATTACK DETECTED
29 attack event(s) recorded · Last checked: 21:33:18
ATTACK EVENTS
29
TOTAL ALERTS
57
UNIQUE IPS
57
TOP OFFENDER
20.57.103.21
REQUESTS PER IP ADDRESS
TRAFFIC STATUS BREAKDOWN
ATTACK
Normal
ALERT LOG – ALL EVENTS
IP ADDRESS
REQUEST COUNT
STATUS
TIMESTAMP

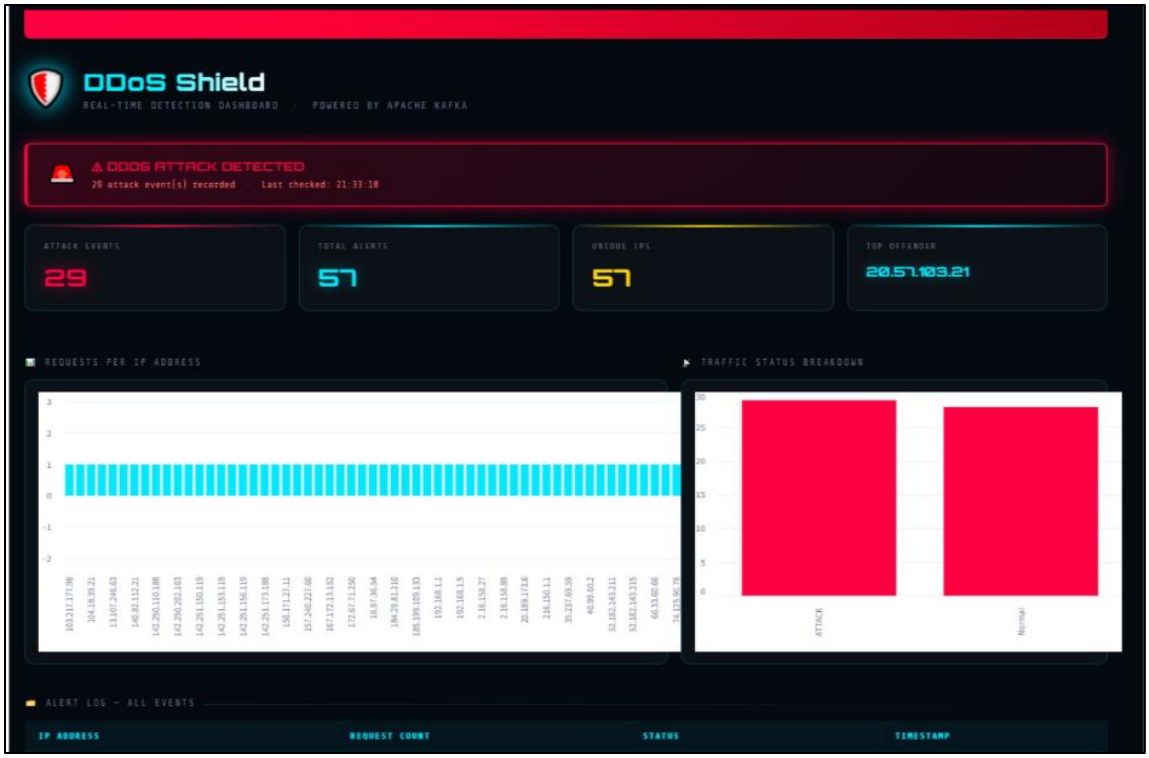
150.171.27.11	18	ATTACK	–
146.190.197.137	9	ATTACK	–
151.101.141.91	2	Normal	–
104.248.110.118	6	ATTACK	–
160.79.104.10	2	Normal	–
40.99.60.2	2	Normal	–
185.199.109.154	2	Normal	–
52.98.61.34	3	Normal	–
185.199.109.133	2	Normal	–
18.97.36.54	12	ATTACK	–
66.33.60.66	2	Normal	–
142.250.186.238	4	Normal	–
74.125.90.67	8	ATTACK	–
142.250.202.163	2	Normal	–
142.251.155.119	4	Normal	–
140.82.113.25	6	ATTACK	–
142.250.110.188	4	Normal	–
52.123.128.14	16	ATTACK	–
74.125.90.78	14	ATTACK	–
2.16.158.27	41	ATTACK	–
52.182.143.215	10	ATTACK	–
4.213.25.241	3	Normal	–
103.217.177.98	82	ATTACK	–
142.251.157.119	1	Normal	–

5.3 Data Persistence Validation

All events sent by the producer were successfully stored in both MySQL and PostgreSQL.

5.4 DDoS Detection Validation

Simulated high traffic from a single IP (e.g., 200 requests in 5 seconds) triggered an alert within 2 seconds. The dashboard displayed the alert in real time.

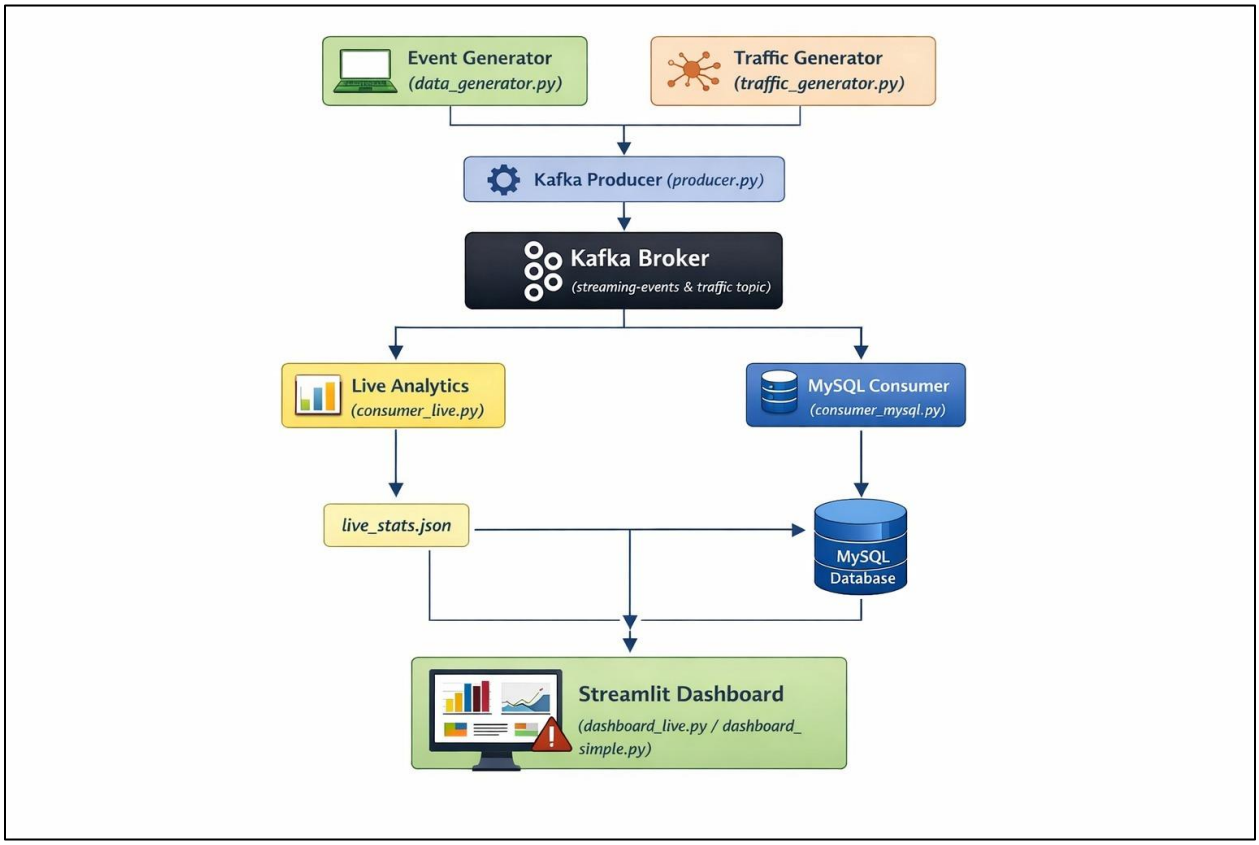


6. Challenges & Solutions

Challenge	Solution
IPv6 vs IPv4 issues with Kafka on Windows	Forced IPv4: os.environ['KAFKA_CLIENT_DNS_LOOKUP'] = 'use_all_dns_ips' and used 127.0.0.1 instead of localhost.
High memory usage with unbounded event buffering	Implemented batched inserts (buffer size 50-100) and periodic flush.
Real-time stats thread conflict	Used threading.Thread(daemon=True) so stats printing does not block event consumption.
DDoS simulation causing consumer lag	Increased partitions to 3 and ran multiple consumers in the same group for parallel processing.

Challenge	Solution
Streamlit dashboard not updating	Used st.cache_data and time.sleep(3) in a loop with st.rerun() (or used st.empty() containers).
MySQL connection drops	Added reconnection logic in mysql_handler.py (retry on failure).

7. System Architecture:



8. Conclusion & Future Work

8.1 Conclusion

The project successfully implements a complete event streaming pipeline using Apache Kafka and Python, extended with DDoS detection capabilities. All planned phases (producer, live consumer, storage consumers, dashboard) were completed. The system demonstrates real-time analytics, persistent storage, attack detection, and interactive visualisation. It validates key Kafka concepts: topics, consumer groups, offsets, batching, and fault tolerance.

8.2 Future Work

- **Deploy to cloud** (AWS MSK, Confluent Cloud) for scalability.
- **Use Avro/Protobuf** with Schema Registry for schema evolution.
- **Implement exactly-once processing** using Kafka transactions.
- **Add machine learning** for anomaly detection (beyond simple threshold).
- **Extend dashboard** with alert history and real-time maps.
- **Integrate with SIEM** (Splunk, ELK) for production security monitoring.

9. References

- Apache Kafka Documentation. (2024). https://kafka.apache.org/documentation/
- kafka-python library. https://kafka-python.readthedocs.io/
- Streamlit Documentation. https://docs.streamlit.io/
- PostgreSQL JSONB. https://www.postgresql.org/docs/current/datatype-json.html
- https://github.com/malayka-ibrar/Live-Event-Streaming-Real-Time-Event-Analytics-Using-Apache-Kafka.git
- https://github.com/Areeba13y/Live-Event-Streaming-Using-Apache-Kafka.git